MetroInsights Phase 2 - Complete Codebase Teaching Guide

Purpose and source version

This index links the source lessons for the checkout at revision `4405d167fc323f33be76e2c7f6614fd21cc574a8` (`main`), audited 2026-10-14. The repository is the authority for behavior. Read in number order for a guided course; use the file/function map to jump to code in an editor. Full descriptions and scope limits are in `00_Audit_and_Coverage.md`.

This is a source teaching series, not proof that operational assumptions are correct or that the demand model is accurate on observed ridership. The current forecaster is trained and evaluated on synthetic generated data. Capacity/fleet/headway and line-share values are configurable project scenarios. Travel durations are graph estimates. The app does not have live DMRC ridership, train dispatch, onboard passenger state, or an actual collision/operations simulator.

Volume guide

PDF	Contents	Best use
`01_Startup_Routes_and_Request_Flow.pdf`	Flask entrypoint, app import/route registration, parsing helpers, landing/dashboard/legacy result routes, request flow	Begin here to understand how a browser request enters Python
`02_API_Prediction_Deep_Walkthrough.pdf`	`/api/predict`, station/metrics endpoints, frontend fetch contract, station schedule return structure	Follow the primary API and response schema
`03_Forecasting_and_Model_Training.pdf`	Runtime features/inference/profile, offline model preparation/training/metrics/artifacts	Understand what the ML prediction actually does
`04_Calendar_Data_and_Synthetic_Generation.pdf`	Operating window, date/occasion context, cleaning, metadata, synthetic data and line split	Trace data creation and scenario assumptions
`05_Scheduling_and_Travel_Graph.pdf`	Capacity/headway scenario, line splits, topology, weighted graph, Dijkstra	Learn scheduling formulas and destination routing
`06_Frontend_Templates_and_JavaScript.pdf`	Three Jinja templates, all browser JS, charts, controls, fetch, theme and animation	Match browser behavior to backend JSON and DOM IDs
`07_Styles_Configuration_and_Legacy_Code.pdf`	CSS, Procfile, dependencies, Git/editor config, project/data docs, old generators	Learn appearance, deployment and compatibility files
`08_End_to_End_Execution_Trace.pdf`	Pipeline/startup/request traces, failure matrix, security and integrated interview practice	Connect the separate subsystems

File -> lesson -> page index

Page numbers below are the printed PDF pages. Source appendices have exact source line ranges in their headings; their page spans below help locate them. Lesson pages precede appendices.

Source path(s)	Lesson or content	Volume and pages
`run.py`, `app/__init__.py`, `app/routes.py` startup, parsing, landing, index, legacy result	Startup and route registration; helper walkthroughs	Vol 01 pp. 1-7; full relevant route source pp. 5-7
`app/routes.py` `api_predict`, `api_stations`, `api_model_metrics`; `dashboard.js` `fetchPrediction`; station optimizer response	API and response contracts	Vol 02 pp. 1-9; API source pp. 6-7; fetch pp. 8-9
`app/forecasting.py` `get_station_info`, `predict_hourly_footfall`, `predict_footfall_with_minute_interpolation`, `predict_station_full_day_profile`, `get_model_evaluation_summary`	Runtime feature row, prediction, interpolation, profile, metrics	Vol 03 pp. 1-11; source pp. 7-8

Source path(s)	Lesson or content	Volume and pages
`pipeline/03_train_and_evaluate_forecaster.py` `prepare_annual_data`, `compute_metrics`, `train_and_evaluate_model`, main guard	Feature preparation, split, estimator, metrics, artifact output	Vol 03 pp. 1-11; source pp. 9-11
`app/operating_hours.py` `is_service_operating`; `app/calendar_context.py` `_occasion`, `get_calendar_context`, `OCCASIONS`	Window boundary, holiday/workday classification	Vol 04 pp. 1-12; source pp. 5-6
`pipeline/01_audit_and_clean_data.py` `clean_stations`, `create_operations_dataset`, data constants	Cleaning and operational assumptions	Vol 04 pp. 1-12; source pp. 7-9
`pipeline/02_generate_calibrated_dataset.py` `build_line_distribution_model`, `compute_station_weights`, `generate_annual_footfall_dataset`, factors	Share/model setup, synthetic records and schema	Vol 04 pp. 1-12; source pp. 10-12
`app/scheduling_optimizer.py` `is_peak_hour`, `get_interchange_line_split`, `round_to_practical_headway`, `optimize_line_schedule`, `optimize_station_schedule`	Worked demand/headway/capacity scenario	Vol 05 pp. 1-12; source pp. 6-8
`app/network_topology.py` `LINES`, `INTERCHANGES`, aliases, penalties	Declarative network and transfer data	Vol 05 pp. 1-12; source pp. 9-10
`app/travel_graph.py` `_normalize`, `_load_stations`, `_is_placeholder_coord`, `_haversine_km`, `_match_topology_to_canonical`, `build_graph`, `shortest_times_from`	Coordinate weighting, line-expanded graph, Dijkstra	Vol 05 pp. 1-12; source pp. 11-12
`app/templates/index.html`, `landing.html`, `result.html`	Template data and DOM contract	Vol 06 pp. 1-20; sources pp. 6-13
`app/static/js/dashboard.js` `getLineColor`, `initControls`, `updateLineSelectOptions`, `fetchPrediction`, `renderDashboard`, `renderDemandChart`, `renderInterchangeBreakdown`, `renderTravelTimes`, `filterAndDisplayTravelTimes`	All dashboard browser logic	Vol 06 pp. 1-20; source pp. 14-18
`app/static/js/landing.js` `drawPreviewChart`, `initScrollMetro`, `initReveals`, `initMobileNavigation`; `theme.js` `applyTheme`	Landing animation/reveals/nav and theme/chart updates	Vol 06 pp. 1-20; sources pp. 19-20
`app/static/css/dashboard.css`, `dashboard-polish.css`, `landing.css`, `metro-theme.css`	Tokens, component groups, media rules and cascade	Vol 07 pp. 1-20; source spans pp. 5-14
`Procfile`, `requirements.txt`, `.gitignore`, `.vscode/launch.json`, `.vscode/settings.json`, `run.py`	WSGI/deploy/runtime dependency/editor configuration	Vol 07 pp. 1-20; source pp. 15-16
`csvFilesGenerator/footfall.py`, `compartmentcapacity.py`, `howtorun.txt`	Legacy synthetic generator and capacity writer	Vol 07 pp. 1-20; source pp. 16-18
`read.md`, `DATA_AUDIT.md`	User docs, data claims and provenance limits	Vol 07 pp. 1-20; source pp. 18-20
all pipeline/runtime modules and browser/request path	Integrated pipeline, startup, request, failure/security and interview trace	Vol 08 pp. 1-7; API function excerpt pp. 6-7

Tracked repository inventory (all 45 files)

Entry, runtime and configuration

`run.py`; `Procfile`; `requirements.txt`; `.gitignore`; `.vscode/launch.json`; `.vscode/settings.json`; `app/__init__.py`; `app/routes.py`; `app/operating_hours.py`; `app/calendar_context.py`; `app/forecasting.py`; `app/scheduling_optimizer.py`; `app/network_topology.py`; `app/travel_graph.py`.

Offline pipeline and legacy scripts

`pipeline/01_audit_and_clean_data.py`; `pipeline/02_generate_calibrated_dataset.py`; `pipeline/03_train_and_evaluate_forecaster.py`; `csvFilesGenerator/footfall.py`; `csvFilesGenerator/compartmentcapacity.py`; `csvFilesGenerator/howtorun.txt`.

UI

`app/templates/index.html`; `app/templates/landing.html`; `app/templates/result.html`; `app/static/js/dashboard.js`; `app/static/js/landing.js`; `app/static/js/theme.js`; `app/static/css/dashboard.css`; `app/static/css/dashboard-polish.css`; `app/static/css/landing.css`; `app/static/css/metro-theme.css`; `app/static/animation.gif`; `app/static/dmrc_logo.png`.

Data, models and project documentation

`data/raw/DELHI_METRO_DATA.csv`; `data/raw/metro_lines.csv`; `csvFiles/DELHI_METRO_DATA.csv`; `csvFiles/metro_lines.csv`; `data/cleaned/stations_cleaned.csv`; `data/cleaned/metro_line_operations.csv`; `data/cleaned/station_typologies.json`; `data/synthetic/line_distribution_model.json`; `models/footfall_forecaster.joblib`; `models/forecaster_metadata.joblib`; `models/evaluation_metrics.json`; `read.md`; `DATA_AUDIT.md`.

Important distinctions and known issues

• Generated `data/synthetic/dmrc_calibrated_footfall.csv` and `csvFiles/dmrc_footfall_data.csv` are ignored/untracked; source is the generator pipeline. Serving loads joblib artifacts and compact station/config data, while training reads the large generated CSV.

• Tracked `.joblib`, PNG, and GIF files are binary. Their format, purpose and code producer/consumer are explained, but they are not printed as source lines. The 130 MB ignored CSVs, virtualenv, bytecode, caches and Git internals are excluded from source lessons and are explicitly documented.

• Existing repository has no tracked automated tests/test configuration, CI workflow, database, lockfile, or Dockerfile.

• Exactly 23:30 is closed under `is_service_operating` because the end comparison is exclusive, despite the phrase `05:00-23:30` often reading as inclusive.

• In the scheduling calculation, practical rounding can produce a trains/hour figure above the pre-rounding fleet clamp; volume 05 explains the numerical example as a potential bug/risk.

• Browser dynamic table output uses some `innerHTML` interpolations; volume 06 marks the injection risk and recommends text nodes.

• Synthetic test metrics do not demonstrate accuracy on actual observed passengers. Topology source claims, operating assumptions and line distribution must not be presented as independently verified unless separately sourced.

How to study

1. Read the volume's explanation before opening the source appendix. 2. Open the exact revision's file and follow the printed source line range; code excerpts preserve consecutive lines. 3. Trace one concrete input through variables and contracts before memorizing names. 4. Explain verified current behavior first, then label risks and proposals separately. 5. Try the exercise without reading the solution, then validate it against the code.

Coverage and honest limits

All 45 tracked files are inventoried. Runtime/pipeline functions and browser handlers are mapped above and have consecutive exact source appendices in the relevant volumes. Data schemas, CSS selector groups, binary assets, configuration and docs receive format-/block-level treatment. This series is not a claim that each punctuation mark in 975 lines of CSS or every repeated markup/style declaration has an individual prose sentence; CSS is taught by meaningful selector groups and cascade behavior. Nor does source coverage verify externally claimed data provenance or operational accuracy. Untracked generated CSVs and environment/dependency internals are excluded for the reasons stated above.